

Practica 3

1. Se dispone de un puente por el cual puede pasar un solo auto a la vez. Un auto pide permiso para pasar por el puente, cruza por el mismo y luego sigue su camino.
 - a. ¿El código funciona correctamente?
Justifique su respuesta.
 - b. ¿Se podría simplificar el programa? ¿Sin monitor? ¿Menos procedimientos? ¿Sin variable condition? En caso afirmativo, rescriba el código.
 - c. ¿La solución original respeta el orden de llegada de los vehículos? Si rescribió el código en el punto b), ¿esa solución respeta el orden de llegada?

a_ El código funciona correctamente. Cuando un proceso auto entra al puente, si la $cant = 0$, incrementa la variable $cant$ y cruza el puente, sino se encola. Cruza el puente, invoca el proceso para salir, resta la variable $cant$ para habilitar el puente y despierta a un auto de la cola (si es que hay).

b_ Si, se podría simplificar de la siguiente forma

```
Monitor Puente {  
    Procedure cruzarPuente () {  
        "El auto cruza el puente"  
    }  
}  
  
Process Auto [1: 1..M] {  
    Puente.cruzarPuente();  
}
```

Como el monitor ya tiene exclusion mutua, cuando proceso invoca al procedure `cruzarPuente`, ningún otro proceso auto puede invocarlo.

c_ No, no respeta el orden de llegada porque cuando llegan procesos, todos compiten para invocar al procedure entrarPuesto(). No, esta solución tampoco respeta el orden de llegada.

2. Existen N procesos que deben leer información de una base de datos, la cual es administrada por un motor que admite una cantidad limitada de consultas simultáneas.
 - a. Analice el problema y defina qué procesos, recursos y monitores/sincronizaciones serán necesarios/convenientes para resolverlo.
 - b. Implemente el acceso a la base por parte de los procesos, sabiendo que el motor de base de datos puede atender a lo sumo 5 consultas de lectura simultáneas.

a_ Un proceso lector, un monitor motor, una variable permanente que limite la consultas y la variable condición cola.

b_

```
Process consulta[1..N] {  
    Motor.solicitarAcceso();  
    //Hacer consulta  
    Motor.devolverAcceso();  
}  
  
Monitor motor {  
    cond cola;  
    int cant = 0;  
  
    procedure solicitarAcceso () {  
        while (cant > 4) {  
            wait(cola);  
        }  
        cant = cant + 1;  
    }  
}
```

```

}

procedure devolverAcceso() {
    cant = cant - 1;
    signal(cola);
}

}

```

3. Existen N personas que deben fotocopiar un documento. La fotocopidora sólo puede ser usada por una persona a la vez. Analice el problema y defina qué procesos, recursos y monitores serán necesarios/convenientes, además de las posibles sincronizaciones requeridas para resolver el problema. Luego, resuelva considerando las siguientes situaciones:
 - a. Implemente una solución suponiendo no importa el orden de uso. Existe una función Fotocopiar() que simula el uso de la fotocopidora.
 - b. Modifique la solución de (a) para el caso en que se deba respetar el orden de llegada.
 - c. Modifique la solución de (b) para el caso en que se deba dar prioridad de acuerdo con la edad de cada persona (cuando la fotocopidora está libre la debe usar la persona de mayor edad entre las que estén esperando para usarla).
 - d. Modifique la solución de (a) para el caso en que se deba respetar estrictamente el orden dado por el identificador del proceso (la persona X no puede usar la fotocopidora hasta que no haya terminado de usarla la persona X-1).
 - e. Modifique la solución de (b) para el caso en que además haya un Empleado que le indica a cada persona cuando debe usar la fotocopidora.

- f. Modificar la solución (e) para el caso en que sean 10 fotocopadoras. El empleado le indica a la persona cuál fotocopadora usar y cuándo hacerlo

3_ Proceso persona, monitor Impresora

a_

```
Monitor Impresora {  
    procedure usarImpresora() {  
        Fotocopiar();  
    }  
}  
  
Process Persona[1..N] {  
    Impresora.usarImpresora;  
}
```

b_

```
Monitor Impresora {  
    bool libre = true;  
    cond espera[N];  
    int idAux, esperando = 0;  
    colaOrdenada cola;  
  
    procedure pasar(idP: in int) {  
        if (!libre) {  
            push(cola, idP);  
            esperando++;  
            wait(espera[idP]);  
        } else {  
            libre = false;  
        }  
    }  
  
    procedure salir() {  
        if (esperando > 0) {  
            pop(cola, idAux);  
        }  
    }  
}
```

```

        esperando--;
        signal(espera[idAux]);
    } else {
        libre = true;
    }
}
}

```

```

Process Persona[id = 1..N] {
    Impresora.pasar(id);
    Fotocopiar();
    Impresora.salir();
}

```

c_

```

// El insertar me inserta por edad

```

```

Monitor Impresora {
    bool libre = true;
    cond espera[N];
    int idAux, esperando = 0;
    colaOrdenada fila;

```

```

    procedure pasar(idP, edad: in int) {
        if (!libre) {
            insertar(fila, idP, edad);
            esperando++;
            wait(espera[idP]);
        } else {
            libre = false;
        }
    }

```

```

    procedure salir() {
        if (esperando > 0) {
            esperando--;
            sacar(fila, idAux);
            signal(espera[idAux]);
        }
    }

```

```

        } else {
            libre = true;
        }
    }
}

Process Persona[1..N] {
    int edad = getEdad();
    Impresora.pasar(id, edad);
    Fotocopiar();
    Impresora.salir();
}

```

d_

```

Monitor Impresora {
    cond espera[N];
    int proximo = 0;

    procedure solicitarImpresora(idP: in int) {
        if (idP != proximo) {
            wait(espera[idP]);
        }
    }

    procedure dejarImpresora(idP: in int) {
        proximo++;
        signal(espera[proximo]);
    }
}

Process Persona[0..N-1] {
    Impresora.solicitarImpresora(id);
    Fotocopiar();
    Impresora.dejarImpresora(id);
}

```

e_

```
monitor Fotocopiadora {

    cond cola, llegoCliente, terminado;
    int esperando = 0;

    procedure pedirEntrar() {
        esperando++;
        signal(llegoCliente);
        wait(cola);
    }

    procedure permitirAcceso() {
        if (esperando == 0) wait(llegoCliente);
        esperando--;
        signal(cola);
        wait(terminado);
    }

    procedure salir() {
        signal(terminado);
    }
}

process Empleado {
    while (true) Fotocopiadora.permidirAcceso();
}

process Persona [id: 0 to N-1] {
    Fotocopiadora.pedirEntrar();
    Fotocopiar();
    Fotocopiadora.salir();
}
```

e_

```

monitor Fotocopiadora {

    cond cola, llegaCliente, terminado;
    int esperando = 0;
    cond impresoras[N];
    queue colaIDs;

    procedure pedirEntrar() {
        esperando++;
        push(colaIDs, id)
        signal(llegaCliente);
        wait(cola);
    }

    procedure permitirAcceso() {
        if (esperando == 0) wait(llegaCliente);
        esperando--;
        pop(colaIDs, aux)

        signal(cola);
        wait(terminado);
    }

    procedure salir() {
        signal(terminado);
    }
}

process Empleado {
    while (true) Fotocopiadora.permidirAcceso();
}

process Persona [id: 0 to N-1] {
    Fotocopiadora.pedirEntrar();
    Fotocopiar();
    Fotocopiadora.salir();
}

```


f_

```
monitor Fotocopiadora {

    cond terminado, cola, llegoCliente, terminado;
    bool impresorasLibres[10] = ([10] , true);
    int esperando = 0, idImpresora;

    procedure pedirEntrar(idImp: out int) {
        esperando++;
        signal(llegoCliente);
        wait(cola);
        idImp = idImpresora;
    }

    procedure permitirAcceso() {
        int i = 0;

        if (esperando == 0) wait(llegoCliente);
        esperando--;

        while (impresorasLibres[i] == false) {
            i++;
            if (i == 10) {
                i = 0;
                wait(terminado);
            }
        }
        idImpresora = i;
        impresorasLibres[i] = false;

        signal(cola);
    }

    procedure salir(idImp: in int) {
        impresorasLibres[idImp] = true;
        signal(terminado);
    }
}
```

```

    }
}

process Empleado {
    while (true) Fotocopiadora.permitirAcceso();
}

process Persona [id: 0 to N-1] {
    int idImp;

    Fotocopiadora.pedirEntrar(idImp);
    Fotocopiar();
    Fotocopiadora.salir(idImp);
}

```

4. Existen N vehículos que deben pasar por un puente de acuerdo con el orden de llegada.
 Considere que el puente no soporta más de 50000kg y que cada vehículo cuenta con su propio peso (ningún vehículo supera el peso soportado por el puente).

```

Monitor Puente {
    bool libre = true;
    cond espera;
    int esperando = 0;
    double pesoActual = 0;

    procedure pasar(double in peso) {
        if (pesoActual + peso > 50000) or (esperando > 0) {
            esperando++;
            push cola, peso;
            wait(espera);
        }
        else {

```

```

    pesoActual += peso;
}
}

procedure salir(double in peso) {
    pesoActual = pesoActual - peso;
    if (esperando > 0) and ((pesoActual+top(cola)) <= 500000 ){
        esperando--;
        pop(cola, peso)
        pesoActual = pesoActual + peso;
        signal(espera)
    }
}

Process Vehiculo[1..N] {
    double peso;
    Puente.pasar(peso);
    //pasa el puente
    Puente.salir(peso);
}

top te trae el valor del primero de la cola

```

5.

6. Existe una comisión de 50 alumnos que deben realizar tareas de a pares, las cuales son corregidas por un JTP. Cuando los alumnos llegan, forman una fila. Una vez que están todos en fila, el JTP les asigna un número de grupo a cada uno. Para ello, suponga que existe una función `AsignarNroGrupo()` que retorna un número "aleatorio" del 1 al 25. Cuando un alumno ha recibido su número de grupo, comienza a realizar su tarea. Al terminarla, el alumno le avisa al JTP y espera por su nota. Cuando los dos alumnos del grupo completaron la tarea, el JTP les asigna un puntaje (el primer grupo en terminar tendrá como nota 25, el segundo 24, y así

sucesivamente hasta el último que tendrá nota 1). Nota: el JTP no guarda el número de grupo que le asigna a cada alumno.

```
alumnos = 50

process Alumno [int id = 0 to 49]{
    Aula.formar(id, grupo)
    //realizarTarea
    Aula.termine(grupo, res)
}

process JTP {
    Aula.asignarGrupos()
    Aula.entregarNota()
}

Monitor aula {
    alumnos [1..50]
    int cant = 0;
    cond vcAlumnos;
    cond vcJTP
    cola colaGrupos
    vectorNotas[1..25];
    vectorGrupos[1..25];
    bool llegaronAlumnos = false;
    vectorEsperaGrupos[] = 0;

    procedure formar (id: in int, grupo: out int){
        cant++;
        if (cant < 50){
            wait (vcAlumnos)
        } else {
            signal(vcJTP);
            llegaronAlumnos = true;
            wait(vcAlumnos);
        }
        grupo = alumnos[id];
    }
}
```

```

procedure asignarGrupo(){
    if (!llegaronAlumnos)
        wait (vcJTP);

    for (int i = 0; i < 50; i++){
        alumnos[i] = asignarNroGrupo();
    }
    signal_all(vcAlumnos);
}

procedure termine(grupo: in int, res: out int){
    if (vectorGrupos[grupo] == 1){
        push (colaGrupos, grupo)
        signal(vcJTP)
    } else {
        vectorGrupos[grupo] = 1
    }
    wait (vectorEsperaGrupos[grupo]);
    res = vectorNotas[grupo];
}

procedure entregarNota(){
    int nroGrupo;
    int nota = 25;
    for (int i = 0; i < 25; i++){
        wait(vcJTP)
        pop (colaGrupos, nroGrupo)
        vectorNotas[nroGrupo] = nota;
        nota--;
        signal_all(vectorEsperaGrupos[nroGrupo]);
    }
}
}

```

7. Se debe simular una maratón con C corredores donde en la llegada hay UNA máquina expendedoras de agua con capacidad para 20 botellas. Además, existe un

repositor encargado de reponer las botellas de la máquina. Cuando los C corredores han llegado al inicio comienza la carrera. Cuando un corredor termina la carrera se dirigen a la máquina expendedora, espera su turno (respetando el orden de llegada), saca una botella y se retira. Si encuentra la máquina sin botellas, le avisa al repositor para que cargue nuevamente la máquina con 20 botellas; espera a que se haga la recarga; saca una botella y se retira. Nota: mientras se reponen las botellas se debe permitir que otros corredores se encolen.

```
Monitor Maraton {
    int cant = 0;
    int esperando = 0;
    cond espera;
    cond maquina;
    boolean libre = true;

    procedure largada() {
        cant++;
        if (cant = C) {
            signal_all(espera);
        } else {
            wait(espera);
        }
    }

    procedure llegadaMaquina() {
        if (!libre) {
            esperando++;
            wait(maquina);
        } else {
            libre = false;
        }
    }
}
```

```

procedure salidaMaquina() {
    if (esperando > 0) {
        esperando--;
        signal(maquina);
    } else {
        libre = true;
    }
}
}

```

```

Monitor Maquina {
    int cantBotellas = 20;
    cond repositor, esperandoBotella;
    bool despertarRepositor = false;

```

```

    procedure reponerMaquina() {
        if (!despertarRepositor) {
            wait(repositor);
        }
        despertarRepositor = false;
        cantBotellas = 20;
        signal(esperandoBotella);
    }

```

```

    procedure usarMaquina() {
        if (cantBotellas = 0) {
            despertarRepositor = true; // para el caso que no haya llegado el rep
ositor
            signal(repositor);
            wait(esperandoBotella);
        }
        cantBotellas--;
    }
}

```

```
Process corredor [id 0..C-1] {  
    Maraton.largada();  
    // corre la maraton  
    Maraton.llegadaMaquina();  
    Maquina.usarMaquina();  
    Maraton.salidaMaquina();  
}  
  
Process repositor {  
    while(true) {  
        Maquina.reponerMaquina();  
    }  
}
```